

南開大學
Nankai University

南开大学

《互联网数据库开发》实验报告

个人作业 1：Web 前端初探

姓名：卢苑

学号：2413275

学院：密码与网络空间安全学院

2026 年 11 月 6 日

目录

1 实验概述 2

2 任务一：请求分析（GET / POST） 2

2.1 实验环境与准备 . . . 2
2.2 GET 请求分析——搜索联想与筛选 . . . 2
2.2.1 联想词请求 recommend . . . 2
2.2.2 筛选标签请求 filter . . . 3
2.3 POST 请求分析——搜索结果与点赞 . . . 4
2.3.1 搜索结果请求 notes . . . 4
2.3.2 点赞请求 like . . . 6
2.3.3 两个 POST 请求的对比 . . . 8
2.4 任务一小结 . . . 8

3 任务二：jQuery 事件触发 9

3.1 什么是 jQuery . . . 9
3.2 jQuery 的注入方式 . . . 9
3.3 确定笔记卡片对应的选择器 . . . 9
3.4 事件一：隐藏所有笔记卡片 . . . 12
3.5 事件二：为笔记卡片添加背景色 . . . 13
3.6 事件三：统计笔记卡片数量并弹窗提示 . . . 14
3.7 任务二小结 . . . 15

4 任务三：浏览器插件设计与实现 15

4.1 功能构思 . . . 15
4.2 页面结构分析 . . . 15
4.3 文件结构 . . . 17
4.4 关键代码 . . . 18
4.4.1 manifest.json . . . 18
4.4.2 content.js . . . 18
4.4.3 background.js ——处理下载请求 . . . 21
4.5 效果展示 . . . 22
4.6 任务三小结 . . . 23

5 总结与思考 23

5.1 遇到的主要问题 . . . 23
5.2 解决思路 . . . 23
5.3 收获与不足 . . . 23

1 实验概述

本次作业围绕「小红书阅读体验增强」这一主题展开，主要包含三部分内容：

1. 针对小红书网页版的请求进行调研，分别分析其 GET 请求（搜索）与 POST 请求（点赞 / 收藏），记录请求及响应数据包；
2. 使用 jQuery 在小红书页面上触发若干事件，展示交互前后页面状态的变化；
3. 设计并实现一个浏览器插件，将笔记内容一键导出为 Markdown 文件，方便整理保存。

2 任务一：请求分析（GET / POST）

2.1 实验环境与准备

本次实验使用 Edge 浏览器访问小红书网页版（`www.xiaohongshu.com`），并登录账号，以保证所需操作能够正常触发。抓包工具使用 Edge 自带的开发者工具（`F12`），主要在 **网络**（Network）面板下进行操作。

为了避免图片、字体等静态资源请求干扰观察，本次实验对请求类型做了两处调整：

1. 在表头空白处右键，勾选显示” 方法” 列，直接查看每个请求是 GET 还是 POST；
2. 使用面板上方的 **Fetch/XHR** 过滤按钮，只保留接口类请求，过滤掉图片、CSS 等静态资源。

2.2 GET 请求分析——搜索联想与筛选

在小红书搜索框中输入关键词” 猫咪” 的过程中，与预想的不同，观察到浏览器实际上发出了不止一个 GET 请求，而是随着输入过程和结果展示，分别触发了” 输入联想” 和” 结果筛选” 两类接口。

2.2.1 联想词请求 recommend

在输入关键词的过程中，页面会实时向后端请求联想词，用于在输入框下方展示候选搜索词。

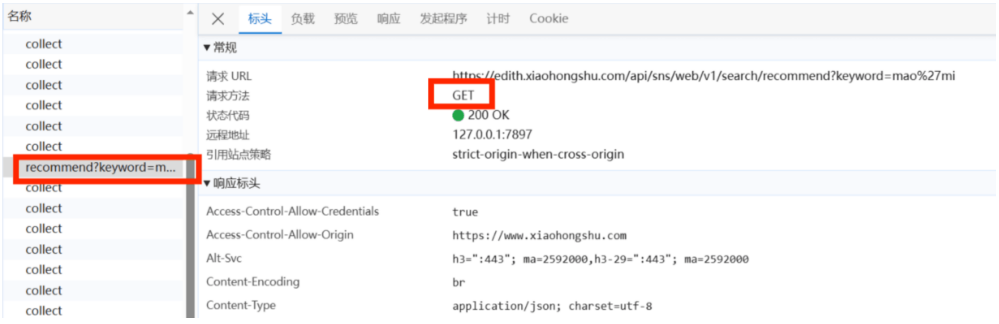


图 2.1: 联想词请求 recommend ——请求头

请求方法为 GET，请求地址为：

Listing 1: recommend 接口请求地址

```
https://edith.xiaohongshu.com/api/sns/web/v1/search/recommend?keyword=mao%27mi
```

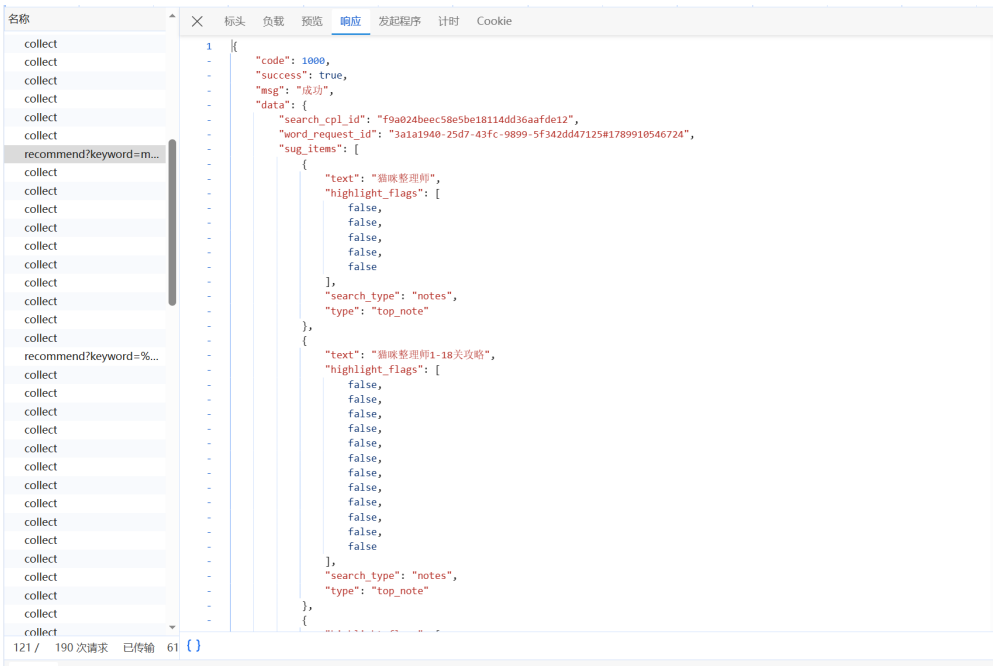


图 2.2: 联想词请求 recommend ——响应数据

从响应数据可以看到，返回的 `sug_items` 字段是一组联想词文本（如”猫咪整理师”“猫咪整理师 1-18 关攻略”），说明这个接口只负责给出候选词，并不返回具体的笔记内容，属于搜索过程中的辅助接口。

2.2.2 筛选标签请求 filter

在关键词确定后，搜索结果页顶部会出现一排可点击的筛选标签（例如”猫咪 + 绘画”“猫咪 + 成都”），这些标签同样通过一个独立的 GET 请求获取。

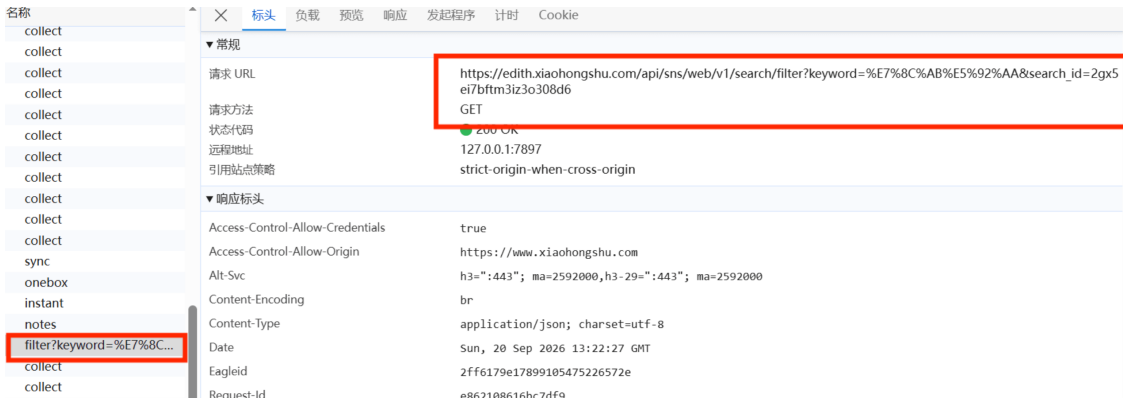


图 2.3: 筛选标签请求 filter ——请求头

请求方法同样为 GET：

Listing 2: filter 接口请求地址

```
https://edith.xiaohongshu.com/api/sns/web/v1/search/filter?keyword=maomi&search_id=...
```

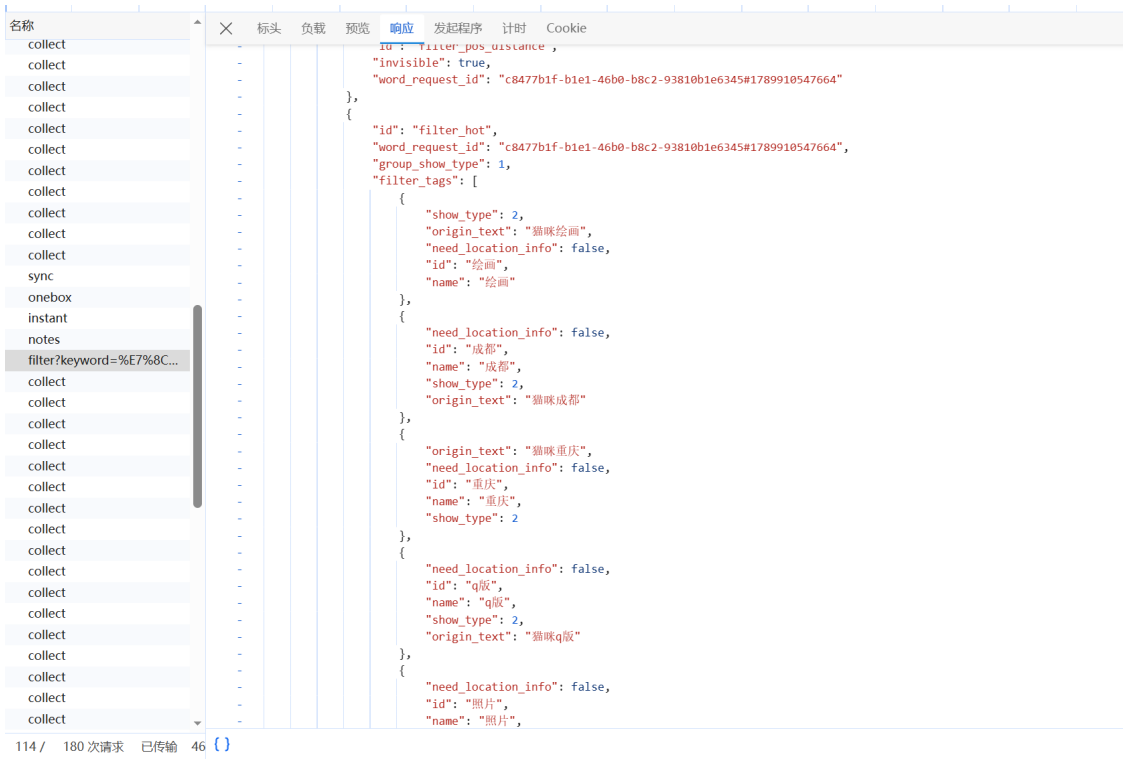

名称
collect
collect
collect
collect
collect
collect
collect
collect
collect
collect
collect
collect
sync
onebox
instant
notes
filter?keyword=%E7%8C...
collect
collect
collect
collect
collect
collect
collect
collect
collect
collect
collect
collect



图 2.6: 搜索结果请求 notes ——请求头

请求地址为:

Listing 3: notes 接口请求地址

```
https://so.xiaohongshu.com/api/sns/web/v2/search/notes
```

查看其请求负载（Payload），发现携带的参数很多:



图 2.7: 搜索结果请求 notes ——负载参数

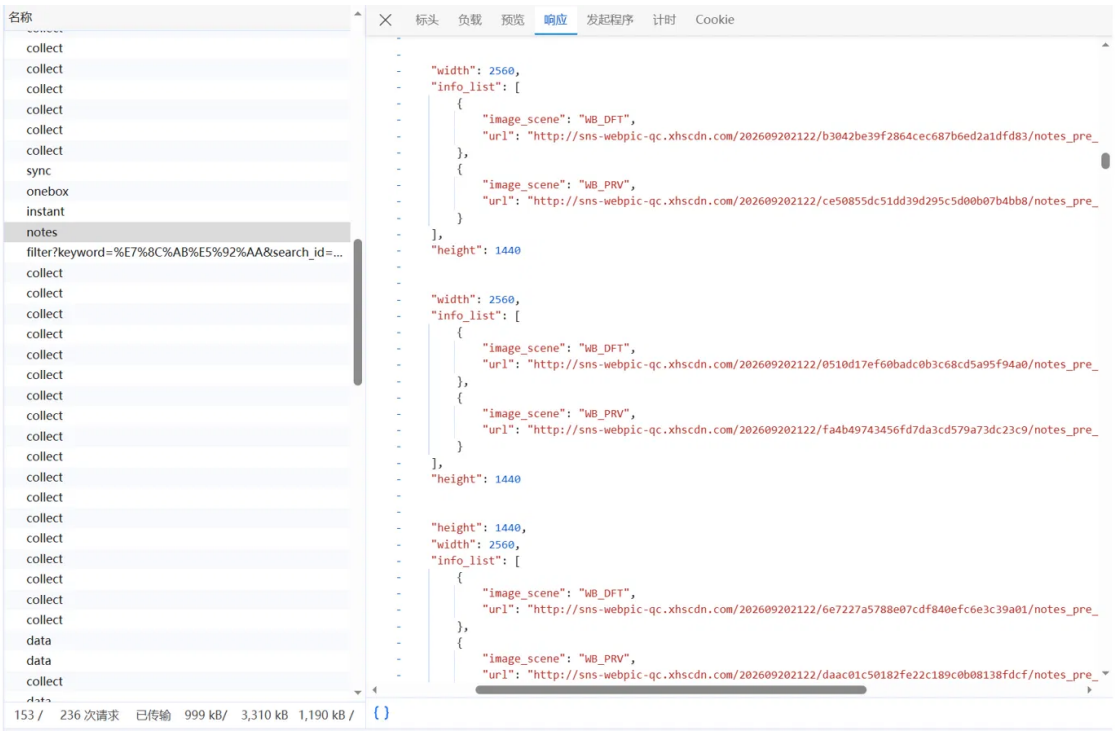


图 2.8: 搜索结果请求 notes ——响应数据（部分笔记信息）

思考：在实验前，我以为”搜索”操作都会使用 GET，实际抓包后发现并非如此，分析原因可能有以下几点：

- **参数复杂**：一次搜索请求要携带关键词、分页、排序方式、筛选条件、支持的图片格式等十余个字段，如果全部拼接进 URL 会非常冗长，用 JSON 请求体传递更清晰，也不受 URL 长度限制；
- **避免缓存**：GET 请求容易被浏览器或中间代理缓存，而搜索结果需要保证实时性（例如点赞数、笔记是否被删除等会随时变化），使用 POST 可以规避这种缓存；
- **降低敏感信息暴露**：GET 参数会明文出现在浏览器历史记录、服务器访问日志中，而用户的搜索关键词属于相对私密的行为数据，放入请求体相对更不容易被直接记录。

这说明实践中的接口设计并不严格遵循”查询用 GET、修改用 POST”的固定模式，还需要权衡性能与数据安全等各方面因素。

2.3.2 点赞请求 like

为了对比 POST 请求在不同业务场景下的用法，进一步抓取了点赞操作对应的请求。（刚好推送了一篇南开大学猫咪的帖子!）

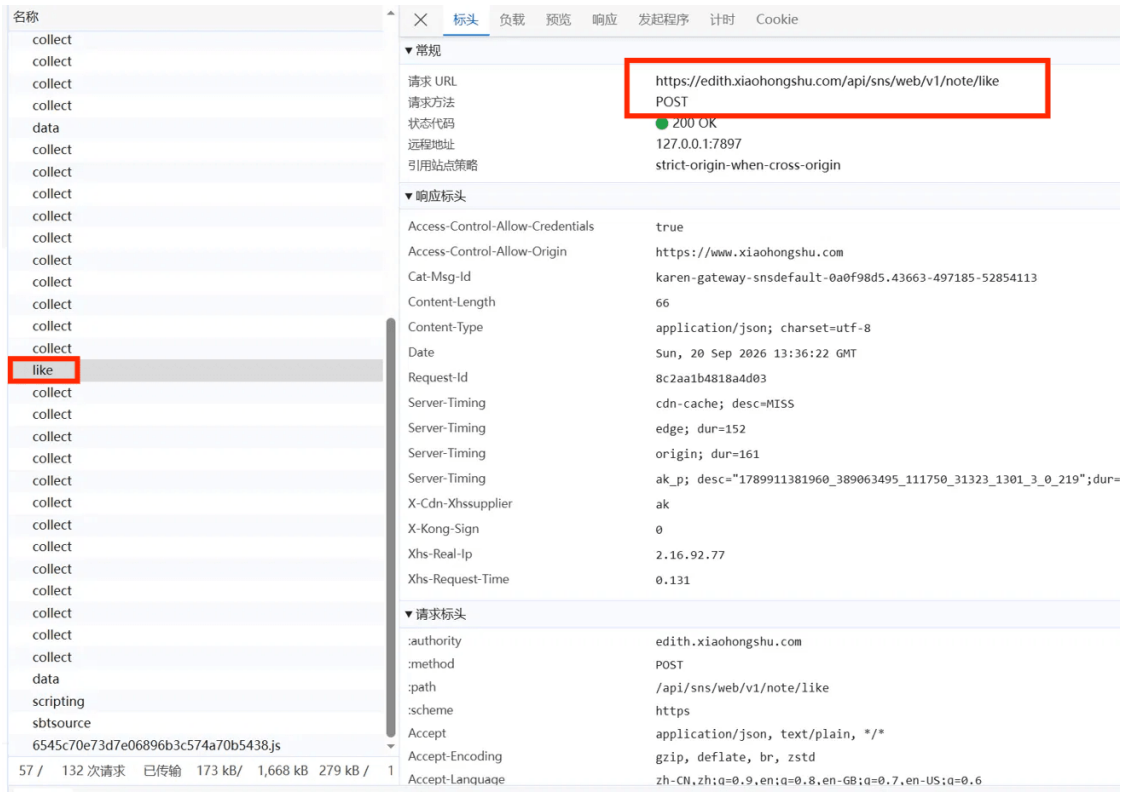


图 2.9: 点赞请求 like ——请求头

Listing 4: like 接口请求地址

```
https://edith.xiaohongshu.com/api/sns/web/v1/note/like
```

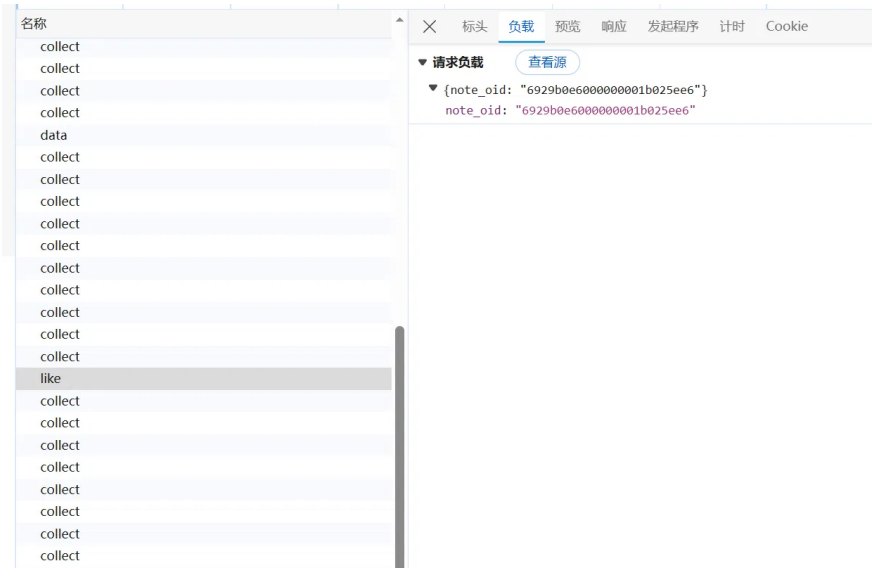


图 2.10: 点赞请求 like ——请求负载

Listing 5: like 接口请求负载（Payload）

```
{
    "note_oid": "6929b0e6000000001b025ee6"
}
```

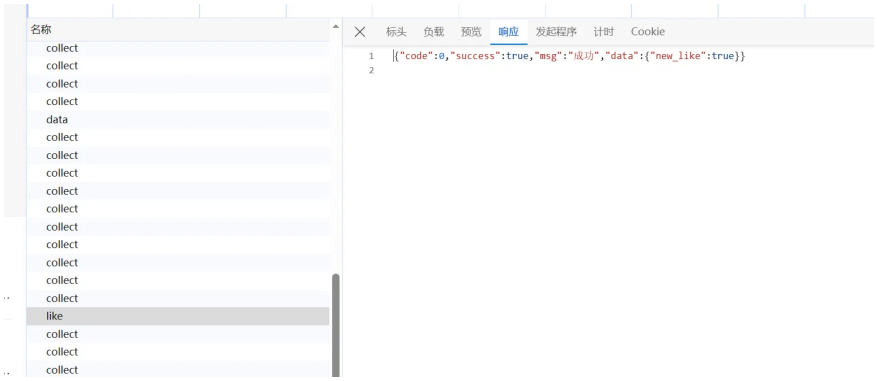


图 2.11: 点赞请求 like ——响应数据

Listing 6: like 接口响应数据

```
{
    "code": 0,
    "success": true,
    "msg": "成功",
    "data": {"new_like": true}
}
```

2.3.3 两个 POST 请求的对比

虽然 `notes` 和 `like` 都使用 POST 方法，但二者的实际作用完全不同，对比如表 1 所示。

表 1: notes 与 like 两个 POST 请求的对比

对比维度	notes（搜索结果）	like（点赞）
操作语义	查询 / 读取数据	修改服务器数据状态
负载复杂度	十余个字段	仅 1 个字段（note_oid）
使用 POST 的原因	参数多、需避免缓存	语义上属于”写操作”
响应内容	大量笔记列表数据	仅返回操作是否成功

可以看出，`notes` 更像用 POST 包装了一个复杂查询，而 `like` 才是传统意义上会改变数据状态的 POST 操作。

2.4 任务一小结

通过本次抓包实验，对”搜索”功能有了更细致的认识，明白了搜索功能实际是由联想词（GET）、筛选标签（GET）、搜索结果（POST）等多个接口协同完成；同时通过与点赞操作（POST）的对比，也体会到 GET / POST 的选择在实际工程中更多取决于参数复杂度、缓存策略、数据敏感性等因素，而非单纯遵循固有规范。

3 任务二：jQuery 事件触发

3.1 什么是 jQuery

jQuery 本质上是一个 JavaScript 的封装库，核心作用是简化”查找页面元素并对其进行操作”这一类任务。网页在浏览器中会被解析成一棵由各个元素组成的树状结构（DOM），jQuery 提供了一个统一的入口函数 `$()`，只需传入类似 CSS 选择器的字符串（如 `.note-item`），就能快速定位到页面上所有符合条件的元素，并对这批元素统一调用诸如 `.hide()`、`.css()` 等现成方法，而不需要像原生 JavaScript 那样逐步编写 `querySelectorAll` 再手动遍历处理。本次实验中用到的三个事件，都是先用 `$('.note-item')` 定位到一批笔记卡片，再调用对应方法进行操作。

3.2 jQuery 的注入方式

由于小红书网页版本身没有加载 jQuery，本次实验选择在浏览器 Console 中动态创建 `<script>` 标签，从 CDN 加载 jQuery 3.7.1：

Listing 7: 通过 Console 注入 jQuery

```
var script = document.createElement('script');
script.src = 'https://cdnjs.cloudflare.com/ajax/libs/jquery/3.7.1/jquery.min.js';
document.head.appendChild(script);
```



图 3.12: 在 Console 中注入 jQuery，`typeof jQuery` 返回 `'function'`，说明注入成功

注：图 3.12 中下方出现的黄色提示 `IM error: LonglinkError:` 通道心跳超时是小红书自身私信模块的长连接超时提示，并非本次 jQuery 注入操作报错。

3.3 确定笔记卡片对应的选择器

在触发事件之前，需要先在页面上找到一张笔记卡片真正对应的 HTML 元素。最初使用 Elements 面板的元素选择器直接点击某张笔记的封面图片，定位到的是 `a.cover.mask.ld` 这一层：

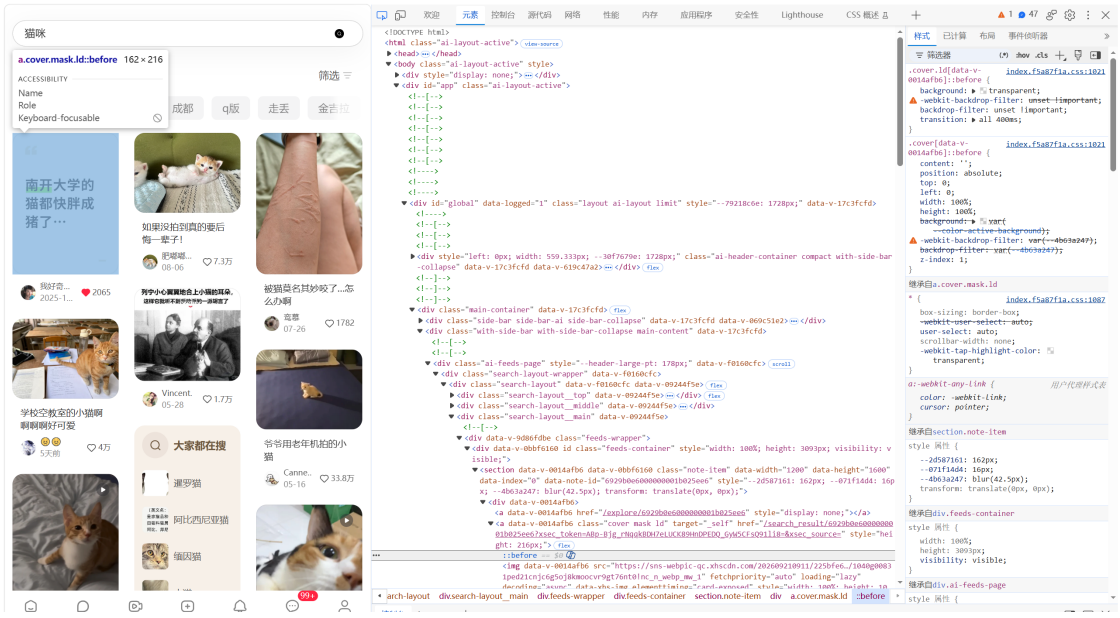


图 3.13: 最初定位到的 a.cover.mask.ld，实际只是卡片内部封面图的链接层

结合 Elements 面板中展开的 HTML 结构进一步观察发现，a.cover.mask.ld 其实是被包裹在一个更外层的 section 标签内部的。从嵌套结构上看，a.cover.mask.ld 只包住了卡片里的封面图片（<img> 标签），并不包含标题文字、作者信息、点赞数等内容；而一张完整的笔记卡片在页面上应该包含”封面图 + 标题 + 作者 + 点赞数” 所有内容，对应到 HTML 结构里，是最外层的 section.note-item 标签。因此可以判断，a.cover.mask.ld 只是卡片内部包裹封面图的一个链接，真正对应卡片整体的是外层的 section.note-item：

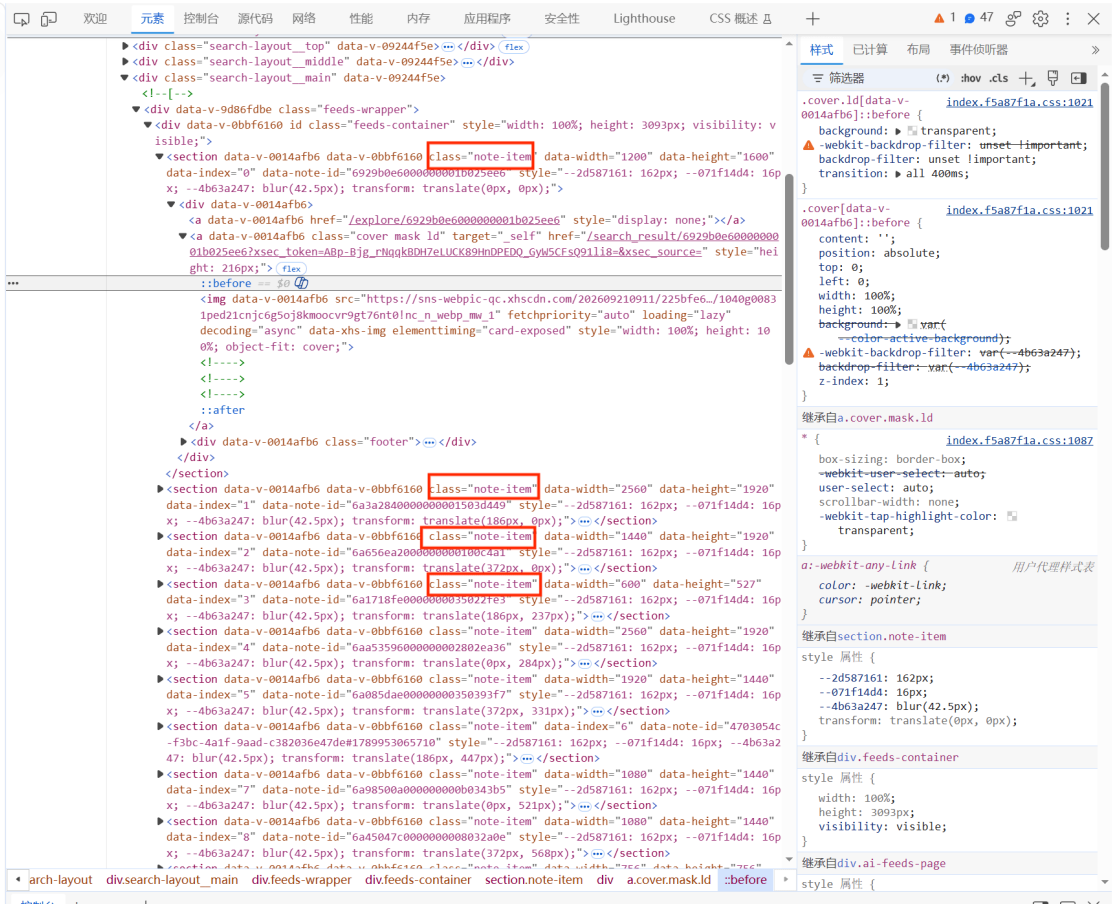


图 3.14: HTML 结构中，`section.note-item` 才是每张笔记卡片对应的外层元素

为了进一步确认`.note-item` 这个 class 是否对应一张笔记卡片，我做了一次交叉验证：在 Console 中执行

Listing 8: 验证选择器命中数量

```
$('.note-item').length
```

统计页面上一共有多少个元素命中这个选择器，并与页面上实际的笔记卡片数量作对比。

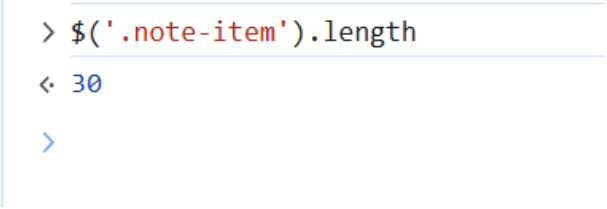


图 3.15: `$('.note-item').length` 返回 30，与当前页面已加载的笔记卡片数量吻合

返回结果为 30，与当前搜索结果页面（已通过滚动加载出的笔记数量）一致，验证了此前基于 HTML 结构做出的判断是正确的：`.note-item` 是对应一张完整笔记卡片的选择器。后续三个事件均基于这个选择器进行。

3.4 事件一：隐藏所有笔记卡片



图 3.16: 执行事件前的搜索结果页面

Listing 9: 隐藏所有笔记卡片

```
1 $('.note-item').hide();
```

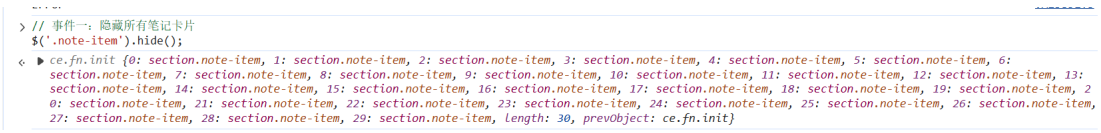


图 3.17: `.hide()` 执行后，Console 返回被选中的 30 个 `section.note-item` 对象



图 3.18: 执行 `.hide()` 后，页面上所有笔记卡片被隐藏，只剩顶部搜索框和筛选标签

对比图 3.16 与图 3.18 可以看出，`.hide()` 实际上是给每个被选中的元素加上了 `display: none` 的行内样式，元素本身并未从 DOM 中删除，因此之后执行 `$('.note-item').show();` 即可让卡片重新显示。这也说明 jQuery 操作 DOM 是可逆的。

3.5 事件二：为笔记卡片添加背景色

在验证完隐藏效果后，先执行 `$('.note-item').show();` 恢复页面，再执行修改样式的语句：

Listing 10: 给笔记卡片添加背景色

```
1 $('.note-item').css('background-color', 'yellow');
```

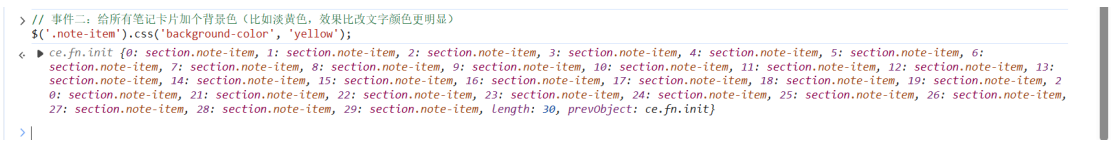


图 3.19: `.css()` 执行后同样返回被选中的 30 个元素



图 3.20: 执行后所有笔记卡片背景变为黄色

.css() 是给选中的所有元素批量添加了行内样式。相比图 3.16 中卡片原本各自的白底样式，图 3.20中每张卡片都被统一覆盖成了黄色背景，复原时，只需将背景色重新设置为空字符串（.css('background-color', '')）即可。

3.6 事件三：统计笔记卡片数量并弹窗提示

Listing 11: 统计元素数量并弹窗提示

```
var count = $('.note-item').length;
alert('当前页面共有 ' + count + ' 张笔记卡片');
```

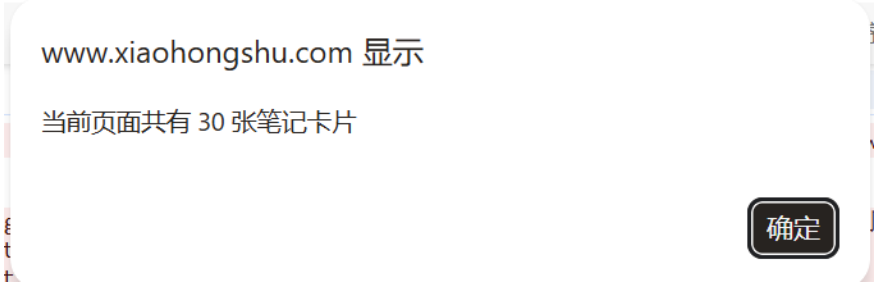


图 3.21: 执行后浏览器弹窗提示”当前页面共有 30 张笔记卡片”

弹窗中统计出的数量与图 3.15 中验证选择器时得到的结果一致。

3.7 任务二小结

完成本任务的过程中我最大的体会是：jQuery 语句本身并不难写，但要注意在页面上”找对元素”。比如本次我通过在 Elements 面板里逐层往上看 HTML 结构，才确认真正代表一张卡片的是外层的 `section.note-item`，并用 `.length` 返回的数量和页面上实际的卡片数做了验证，确认选择器无误后再进行后续操作。

此外我也想到，由于小红书是单页应用，页面内容是前端动态渲染出来的，所有通过 Console 注入的 jQuery 以及执行的 DOM 修改都只是临时生效，刷新页面就会消失，需要重新注入并执行；这也说明如果想要获得持久的功能，需要使用任务三中的浏览器插件，通过 `content_scripts` 在页面加载时自动注入脚本。

4 任务三：浏览器插件设计与实现

4.1 功能构思

我自己平时刷小红书的时候，经常会刷到一些实用的”干货”笔记，比如知识点梳理、教材推荐等，刷到时会点收藏，但收藏夹里的内容比较混乱、难以分类整理，基本上再不会打开第二次，内容也没办法导入自己平时用的笔记软件。所以我希望做一个可以将笔记内容导出为 Markdown 的插件：在详情页加一个悬浮按钮，点击后自动提取当前笔记的标题、正文、话题标签、作者、发布时间等信息，组装成 Markdown 格式，同时复制到剪贴板并下载为本地文件，方便把喜欢的笔记整理保存到 Typora、Obsidian 等常用的 Markdown 笔记软件里。

4.2 页面结构分析

在写提取逻辑之前，需要先逐个确认标题、正文、标签、作者、时间这几块内容在 DOM 里分别对应的选择器。

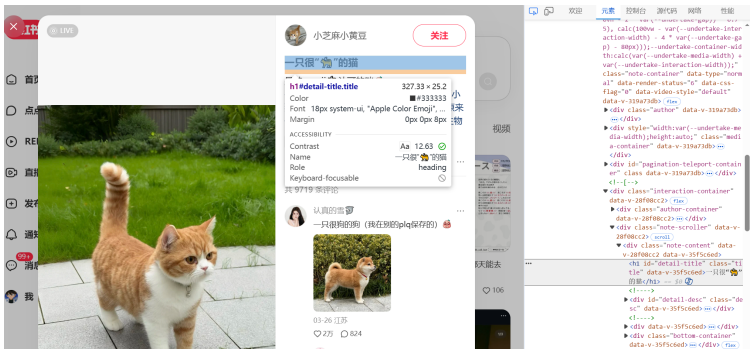


图 4.22: 标题对应元素为 `h1#detail-title`，位于 `.note-content` 容器内

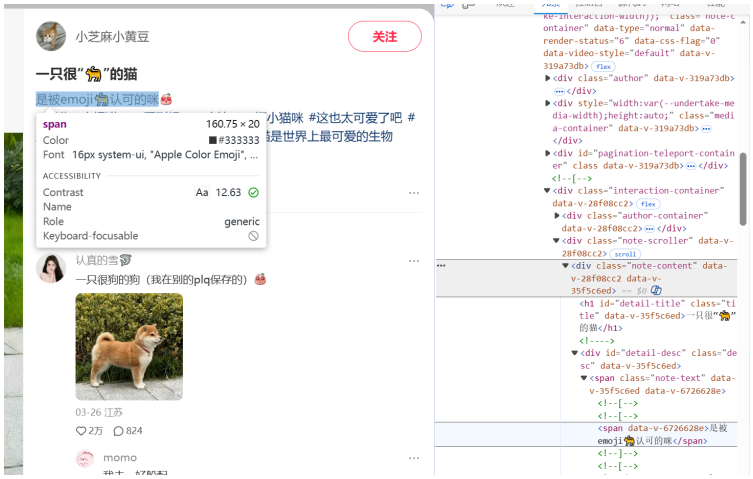


图 4.23: 正文对应元素为 `#detail-desc .note-text`

在查看正文结构时发现一个细节：正文文字和话题标签在同一个 `span.note-text` 容器内部，标签是里面的若干个 `<a class="tag">` 子元素。



图 4.24: 单个话题标签对应 `a.tag`，是 `.note-text` 内部的子元素

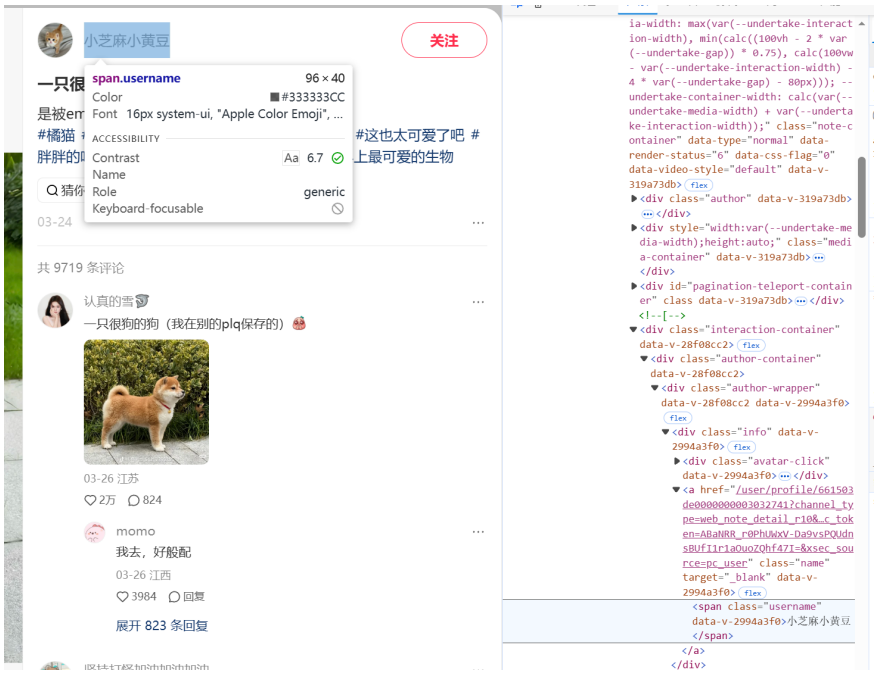


图 4.25: 作者昵称对应 `.author-container .username`



图 4.26: 发布时间/地点对应 `.bottom-left .date`

由于标签是嵌套在正文内部的，提取的时候不能简单地把 `.note-text` 的文字整体取出来，需要先把里面的 `a.tag` 单独摘出来存成标签列表，剩下的部分再作为纯正文，这样导出的 Markdown 里标签可以单独成行，格式会更清晰。

4.3 文件结构

最终插件由三个文件组成：

Listing 12: 插件目录结构

```
LAB1/
|-- manifest.json
|-- content.js
|-- background.js
```

4.4 关键代码

4.4.1 manifest.json

Listing 13: manifest.json

```
{
  "manifest_version": 3,
  "name": "小红书笔记 Markdown 导出",
  "version": "1.0",
  "description": "一键将小红书笔记导出为 Markdown 文件",
  "permissions": ["downloads", "clipboardWrite"],
  "background": {
    "service_worker": "background.js"
  },
  "content_scripts": [
    {
      "matches": ["*://*.xiaohongshu.com/*"],
      "js": ["content.js"],
      "run_at": "document_idle"
    }
  ]
}
```

代码中声明了两个权限：`downloads` 用于把生成的 Markdown 保存成本地文件，`clipboardWrite` 用于把内容复制到剪贴板。另外还多了一个 `background` 字段，指向一个后台常驻脚本，用来处理下载请求（原因见下文）。

4.4.2 content.js

Listing 14: 提取笔记信息

```
function extractNoteData() {
  const titleEl = document.querySelector('#detail-title');
  const descEl = document.querySelector('#detail-desc .note-text');
  const authorEl = document.querySelector('.author-container .username');
  const dateEl = document.querySelector('.bottom-left .date');

  if (!titleEl || !descEl) {
    alert('未能识别笔记内容，请确认当前处于笔记详情页');
    return null;
  }

  const title = titleEl.innerText.trim();
```

```

  // 提取标签 (.note-text 内部的 a.tag)
  const tagEls = descEl.querySelectorAll('a.tag');
  const tags = Array.from(tagEls).map(a => a.innerText.trim());

  // 提取纯正文：克隆一份 .note-text，把里面的标签 <a> 删掉，剩下的就是纯文字
  const descClone = descEl.cloneNode(true);
  descClone.querySelectorAll('a.tag').forEach(a => a.remove());
  const bodyText = descClone.innerText.trim();

  const author = authorEl ? authorEl.innerText.trim() : '未知作者';
  const dateInfo = dateEl ? dateEl.innerText.trim() : '';
  const url = window.location.href;

  return { title, bodyText, tags, author, dateInfo, url };
}
```

代码中对正文和标签进行了拆分：因为标签 <a> 标签是嵌套在正文容器内部的，如果直接取 .innerText，标签文字会和正文混在一起。因此先把整个正文节点 cloneNode(true) 克隆一份，再在副本上把所有 a.tag 元素删掉，剩下的文字就是不含标签的正文，不会影响到页面上原本的真实 DOM。同时用 querySelectorAll('a.tag') 单独把每个标签的文字取出来存成数组，方便后面单独排版。

Listing 15: 组装 Markdown 格式

```
function buildMarkdown(data) {
  let md = `# ${data.title}\n\n`;
  md += `**作者：** ${data.author}\n`;
  md += `**${data.dateInfo}**\n\n`;
  md += `${data.bodyText}\n\n`;
  if (data.tags.length > 0) {
    md += data.tags.map(t => t.startsWith('#') ? t : '#' + t).join(' ') + '\n\n';
  }
  md += `> 来源：${data.url}\n`;
  return md;
}
```

这里把标题用一级标题的语法 # 表示，作者和日期用加粗，正文之后单独留一行放所有标签，最后用 Markdown 的引用语法 > 标注笔记原始链接，方便访问原笔记。标签列表判断是否已经带 # 前缀（t.startsWith('#')）是为了避免存在标签文字本身没有带 # 号导致格式不统一。

Listing 16: 导出：复制剪贴板并触发下载

```
async function exportAsMarkdown() {
  const data = extractNoteData();
  if (!data) return;
```

```

  const markdown = buildMarkdown(data);

  // 复制到剪贴板
  try {
    await navigator.clipboard.writeText(markdown);
  } catch (err) {
    console.warn('剪贴板复制失败', err);
  }

  // 触发下载
  const blob = new Blob([markdown], { type: 'text/markdown;charset=utf-8' });
  const url = URL.createObjectURL(blob);
  const safeTitle = data.title.replace(/[\\/:*?"<>|]/g, '').slice(0, 30) || 'note';

  chrome.runtime.sendMessage({
    type: 'DOWNLOAD_MD',
    url: url,
    filename: `${safeTitle}.md`
  });

  showToast('已导出并复制到剪贴板 ');
}
```

首先用浏览器原生的剪贴板 API 把 Markdown 文本复制出去，方便直接粘贴使用；然后把文本包装成一个 `Blob` 对象，生成一个临时的本地 URL，交给后台脚本去下载。其中 `safeTitle` 这一行是为了防止笔记标题里出现斜杠、问号这些文件名不允许出现的字符（图 4.30 里可以看到，我测试用的这篇笔记标题里还带了一个表情符号，如果不做处理，文件名可能会出问题）。

Manifest V3 规范下，content script 是不能直接调用 `chrome.downloads` 这个 API 的，必须通过消息机制转发给后台的 service worker 来完成下载动作，这也是 `background.js` 文件存在的原因。

Listing 17: 创建悬浮导出按钮

```
function createExportButton() {
  if (document.getElementById('md-export-btn')) return; // 避免重复创建

  const btn = document.createElement('button');
  btn.id = 'md-export-btn';
  btn.innerText = '导出 MD';
  btn.style.cssText = `
    position: fixed; right: 20px; bottom: 100px; z-index: 99999;
    background: #ff2442; color: #fff; border: none; border-radius: 28px;
    padding: 16px 28px; font-size: 18px; font-weight: 600; cursor: pointer;
```

```
    box-shadow: 0 4px 12px rgba(0,0,0,0.25);
  `;
  btn.addEventListener('click', exportAsMarkdown);
  document.body.appendChild(btn);
}

function checkAndToggleButton() {
  const isDetailOpen = document.querySelector('.note-detail-mask') !== null;
  const existingBtn = document.getElementById('md-export-btn');
  if (isDetailOpen && !existingBtn) {
    createExportButton();
  } else if (!isDetailOpen && existingBtn) {
    existingBtn.remove();
  }
}

const observer = new MutationObserver(checkAndToggleButton);
observer.observe(document.body, { childList: true, subtree: true });
checkAndToggleButton();
```

按钮通过判断页面上是否存在 `.note-detail-mask`（即详情弹窗有没有打开）来动态显示或隐藏，配合 `MutationObserver` 监听 DOM 变化实现。这和任务二小结里提到的思路保持一致，因为小红书是单页应用，弹窗的打开关闭都是前端动态渲染出来的，不能固定在页面加载完后执行一次，要持续监听页面结构的变化。

4.4.3 background.js ——处理下载请求

Listing 18: background.js

```
chrome.runtime.onMessage.addListener((message) => {
  if (message.type === 'DOWNLOAD_MD') {
    chrome.downloads.download({
      url: message.url,
      filename: message.filename,
      saveAs: false
    });
  }
});
```

这段代码的目的是监听 content.js 发来的消息，收到 `DOWNLOAD_MD` 类型的消息后调用 `chrome.downloads.downlo` 把文件保存到本地。

4.5 效果展示

将插件文件夹加载到 Edge 扩展管理页面后，插件成功加载，如图 4.27 所示：



图 4.27: 插件成功加载

打开一篇笔记详情页后，右下角会出现一个红色的悬浮按钮：



图 4.28: 笔记详情页右下角出现” 导出 MD” 悬浮按钮

点击按钮后，浏览器下载栏立即出现对应的 `.md` 文件，文件名自动取自笔记标题：



图 4.29: 导出的 Markdown 文件成功下载

打开下载好的文件，可以看到标题、作者、日期、正文、标签（单独成行）、原始链接都被正确提取和排版，和右侧原始笔记内容对照可以确认信息没有遗漏或错位：



图 4.30: 导出的 Markdown 内容（左）与原笔记页面（右）对比

4.6 任务三小结

这次任务三完成的插件，个人认为还比较贴近真实使用需求，实现过程中用到了较多新的技术点，比如 DOM 节点克隆后再删除子元素来拆分正文和标签、Blob 生成临时下载链接、以及 Manifest V3 下 content script 和 background service worker 之间通过消息传递协作完成下载。这些都是我之前没有接触过的内容，加深了我对于前端的应用理解。

5 总结与思考

5.1 遇到的主要问题

问题主要集中在页面结构不稳定：任务二里最初点封面图定位到的是内部链接层 `a.cover.mask.ld`，往上翻几层才找到真正的卡片容器 `section.note-item`；任务三里标签也不是独立区域，而是嵌在正文 `span.note-text` 内部的 `<a>` 标签，不细看嵌套关系很容易选错层级。加上小红书是单页应用，内容靠滚动和弹窗动态加载，如果只在加载完后执行一次会覆盖不到后续新内容，这个问题在任务二结尾就发现了，任务三用 `MutationObserver` 才算真正解决。

5.2 解决思路

选择器定位不准可以靠”点击选中 → 在 Elements 面板往上找父容器 → 用 `.length` 验证数量”的思路，确认选对了选择器再往下写逻辑。

动态内容则可以用 `MutationObserver` 监听 `document.body`，一旦结构变化就重新处理一遍，不管是新加载的卡片还是弹窗的开关状态都能实时捕捉到。

5.3 收获与不足

这次实验让我对网页背后的运作方式有了更具体的认识：一个操作发出的是 GET 还是 POST、为什么这么设计；jQuery 和原生 JS 操作 DOM 的关系；一个能持久生效的插件是怎么通过 `content_scripts` 注入、通过 `manifest.json` 声明权限、通过消息机制在 content script 和 background 之间传递数据的。这些内容以前都只是理论上了解，没真正动手做过。

不足也很明显：选择器基本是写死的具体 class 名，网站一旦改版很可能失效，容错处理也不够，以及标题里表情符号导致文件名异常这种边界情况，都是做的过程中才想到要处理。总的来说这是我第一次完成 Web 前端相关任务，最后做出来的东西是自己实际需要用到的，还比较有成就感！